CSE 102 Assignment 2 Report

1. Introduction

This report presents an overview of a simple game implemented in C, where a player navigates a 2D grid to reach a door. The implementation adheres to the assignment constraints by using only scalar variables, conditional statements, and file handling for game state persistence. The game progresses through multiple program executions, without loops or functions.

2. Implementation Details

2.1 Board Generation and Initialization

The game starts by checking if a file named game_state.txt exists. If the file is present, it loads the stored game state; otherwise, a new board is generated. This is achieved with the following code:

```
dosya = fopen("game_state.txt", "r");

if (dosya == NULL) {
    printf("Generating a new board...\n");
    printf("Enter the width and height of board: ");
    scanf("%d %d", &width, &height);

    int t = time(NULL);
    playerX = t % width;
    playerY = (t / 2) % height;
    doorX = (t / 3) % width;
    doorY = (t / 4) % height;
```

Here, time(NULL) is used to introduce randomness into the initial positions of the player and the door.

2.2 Player Movement

The player moves using WASD keys, where:

- W moves up
- S moves down
- A moves left
- D moves right

The movement logic is handled with conditional statements:

```c
printf("Enter move (WASD): ");
scanf(" %c", &move);

if (move == 'W' || move == 'w') {
    playerY--;
    printf("Player moves up to (%d, %d)\n", playerX, playerY);
} else if (move == 'S' || move == 's') {
    playerY++;
    printf("Player moves down to (%d, %d)\n", playerX, playerY);
} else if (move == 'A' || move == 'a') {
    playerX--;
    printf("Player moves left to (%d, %d)\n", playerX, playerY);
} else if (move == 'D' || move == 'd') {
    playerX++;
    printf("Player moves right to (%d, %d)\n", playerX, playerY);
} else {
    printf("Invalid key, please enter W, A, S, or D\n");
    return 0;
}
```

2.3 Collision Detection

A boundary check ensures that the player does not move outside the grid:

```c
if (playerX < 0 || playerX >= width || playerY < 0 || playerY >= height) {
    printf("You hit the wall!\n");
    return 0;
}
```

If the new position is out of bounds, the program terminates with a message indicating that the player hit the wall.

2.4 Winning Condition

If the player reaches the door, the game congratulates the user and resets the board by deleting the game_state.txt file:

```c
if (playerX == doorX && playerY == doorY) {
    printf("Congratulations! You escaped!\n");
    remove("game_state.txt");
}
```

If the player does not win, the new state is saved back to game_state.txt to continue in the next execution.

```
else {
    dosya = fopen("game_state.txt", "w");
    fprintf(dosya, "%d %d %d %d %d %d", width, height, playerX, playerY, doorX, doorY
    fclose(dosya);
    printf("Game continues...\n");
}
```

3.Output

```
Generating a new board...
Enter the width and height of board: 5 5
Width: 5, Height: 5 Player: (4,2), Door: (1,3)
Enter move (WASD): A
Player moves left to (3, 2)
Game continues...

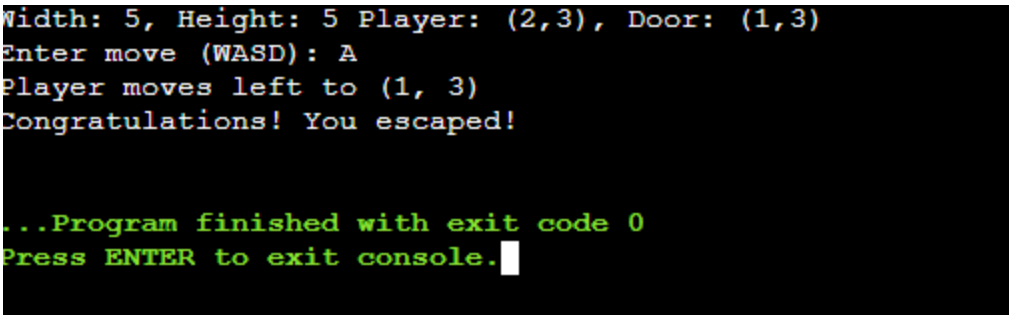

...Program finished with exit code 0
Press ENTER to exit console.
```

```
Width: 5, Height: 5 Player: (3,2), Door: (1,3)
Enter move (WASD): A
Player moves left to (2, 2)
Game continues...


...Program finished with exit code 0
Press ENTER to exit console.
```

```
Width: 5, Height: 5 Player: (2,2), Door: (1,3)
Enter move (WASD): S
Player moves down to (2, 3)
Game continues...


...Program finished with exit code 0
Press ENTER to exit console.
```



4. Conclusion

The game is successfully implemented according to the assignment constraints. It ensures that the player progresses towards the door while maintaining game persistence across executions. The use of file handling, conditional statements, and scalar variables makes the program compliant with the given requirements.